Week 5 Tutorial - Part 2

Conditionals

The if statement

Sometimes we want to check whether a condition has been met - for instance, whether a variable contains a particular value; whether an object exists, whether an element exists within an array. We achieve this using the `if` statement, and the syntax is as follows:

```
if this condition is satisfied, do the following
```

or in actual code:

```
if(condition == TRUE){
  do_this()
  and_this()
  et_c()
}
```

A few notes about setting conditions:

1. When checking the equality of two values - e.g. whether condition is equal to TRUE, we use the double equal-sign operator `x == y`.

2. When checking the inequality of two values we use an exclamation sign followed by an equal sign (literally 'is not equal to') - `x != y`

3. When handling numerical data we can check for the direction of difference using greater than and less than signs `x > y`, `x < y`. These can also be paired with the equals sign e.g. `x >= y` means "is y at least as big as x".

4. To check if a value is within an array we can use the `%in%` operator which checks each element - e.g. `y %in% x`

All of the above would return `TRUE` if the condition is satisfied or `FALSE` otherwise.
Try typing in `1==1`, `1 != 2`, or `2 > 3` in the console.

What does this look like in an `if` statement?

```
x <- 1

if(x == 1){
  print('x is 1')
}

if(x <=2){
  print("x is less than 2")
}

x <- c(1,2,3,4,5)
y <- 1

if(y %in% x){
  print('y is an element of x')
}
```

The else statement

We can use the `else` statement to set a fallback - i.e. code to run if the condition is not true.

For instance:

```
x <- "hello"
y <- "world"

if(x==y){
  print("They're the same!")
} else{
  print("they're not the same!")
}
```

As we will see later, we can use conditionals to control the flow of our code and to assign different values to the same variable.

Loops

One of the most useful things about computers is that they can repeatedly do the same thing very quickly. One of the ways in which we an achieve this is though loops. The name is no coincidence - a loop tells the computer to do a set of tasks and then return to the first task a given number of times, or until a certain condition is met. We will look at both of these types of loop, in that order.

For Loops

Earlier we saw that variables can have multiple elements (vectors), across multiple dimensions (arrays). If we want to perform an action or run a function on each of the elements in a vector or an array we *could* do the following:

```
### Create a vector
my_vector <- c(1,2,3,4,5)

### Multiple each element by 2
my_vector[1] <- my_vector[1] * 2
my_vector[2] <- my_vector[2] * 2
my_vector[3] <- my_vector[3] * 2
my_vector[4] <- my_vector[4] * 2
my_vector[5] <- my_vector[5] * 2
```

But this is very inconvenient for the following reasons:

1. We would have to type everything out five times,
2. or copy and paste the same line and then change the numbers,
3. if want to change the number we multiply by we have to change it 5 times,
4. if we want to add a 6th element we would have to type everything out again,
5. or… you get the picture

Instead, we can write a program that takes each element in the vector and multiplies it by two. This way we only have to write a couple of lines of code, and the upshot is that we can give a vector of any length.

The for statement

To write a loop that goes through each element of a vector we use the `for` statement. The structure of a `for` statement in normal language is:

```
for every element in our vector complete the following instructions until there are
no elements remaining
```

How this is written differs depending on the language, but the structure is always the same. In R, we do the following:

```
for (element in vector){
  my_instructions(element)
}
```

In the above code `element` starts as an empty variable. It then takes the value of each element in the `vector` object sequentially.

The exercise below provides the skeleton of a for loop, adapt it so that it does the following: 1. prints each element of the vector to the console, 2. prints the type of each element to the console, 3. prints each element multiplied by 10 as a character to the console.

```
my_vector <- 1:10

for( element in my_vector){
  ## Enter your code here
}
```

How is this useful?

I agree, in the above case we are first only printing values to the console which is not very useful outside of looking for bugs. Secondly, we already know the numbers that are being entered into the loop, and most people could do the multiplication in their head. But imagine that we had a much larger vector - e.g. of 1 million elements - and we had to do much more complicated operations on it (e.g. take the log or square root). Or for a more applied use case - we might have a data-set with 1000 observations and need to shuffle this 10,000 times and store a summary of each iteration (a procedure called bootstrapping that is often used in statistics). This would be much less trivial!

First let's try storing our outputs.

There are a couple of ways to handle this. The first is much slower than the second which can make a difference when handling large vectors or arrays (e.g. those with 100k + elements).

```
my_vector <- 1:10
my_new_vector <- c() ## Create an empty variable

for(element in my_vector){
my_new_vector <- c(my_new_vector, element * 2)
print(my_new_vector)
}
```

Try running the above code below (type it or copy & paste).

The faster approach is to create an empty vector of a given size and assign each output using indices.

The following does exactly the same as the code above, but instead of assigning the empty variable in our `for` statement the value of each element in the vector, it assigns the position of each element in the vector. We then print the element by indexing that position.

```
n <- 10
my_vector <- runif(n) # This is a function that generates n number of random numbers
my_new_vector <- vector(length=10, mode='numeric')
for(index in 1:n){ ### Here we feed the for loop a new vector containing the numbers
1-n
my_new_vector[index] <- my_vector[index] * 2
}
```

I have introduced two new functions above. The first is `runif()` which just returns a set of `n` random

numbers.

The second is the `vector()` function which produces a vector of `n` zeroes. The `mode` argument tells the function how to represent the zeros. The default is logical which returns `FALSE`. We have changed this to 'numerical' so that it returns zeroes.

This just makes it easier to produce example vectors and we will be going into more detail later. Try running the code below.

Controlling the flow of a loop

It might be that we want to skip over some values within our vector, or stop the loop altogether. The `next` and `break` statements allow us to do this.

next

If `next` is called the loop skips the current iteration and continues - for instance:

```
#Sequentially print 1-10
for(i in 1:10){
    print(i)
}

# Sequentially print 1-10 but skip out 5
for(i in 1:10){
  if(i == 5){
    next
}
print(i)
}
```

break

Break is more extreme and just terminates the loop entirely

```
for(i in 1:1e4){ ## Print out the first 10 elements of 10,000
  if(i <= 10){
    print(i)
  }else{
    break
  }
}
```

While Loops

Sometimes we want to keep running a loop until a condition is reached without worrying about how many times the loop repeats. For this we can use a `while` loop. A very basic example that replicates the behaviour above is the following:

```
i <- 0 # Create a new variable, i, that is equal to 0
while(i <= 10){ # Run the following until i equals 10
  print(i) # Print the current value of i
  i <- i + 1 # add 1 to i
}

#### Another (longer) way to write this is:

run_me <- TRUE
i <- 0
while(run_me){
  print(i)
  if(i >=10){
```

```
    run_me <- FALSE # Loop breaks when i equals 10
  }
}
```

Exercises

Try using what we have done above to make a coin flipping machine that: 1. Flips a coin 100 times, 2. keeps flipping a coin until heads is called.

Hint - you can use the sample() function to select an element from a vector at random.

Exercise 1

Flip a coin 100 times

```
coin <- c('Heads', 'Tails')
for(i in 1:100){
  print(sample(coin,1))
}
```

Exercise 2

Keep flipping a coin until heads is called.

```
# Set up your variables
coin <- c('Heads', 'Tails')
keep_flipping <- TRUE
while(keep_flipping){
  this_flip <- sample(coin, 1) # Choose one element from coin at random
  print(this_flip)
  if(this_flip == "Heads"){
    keep_flipping <- FALSE

  }
}
```

Run this code a few times - how often do you get more than one ‘Tails’ in a row?

Exercise 3: Optional Brain-Teaser

If that was too easy, try writing it so that it stops when three heads are flipped in a row!

```
three_heads <- FALSE
coin <- c("Heads", "Tails")
head_count <- 0
while(!three_heads){
  this_flip <- sample(coin, 1) #Pick a random element from coin
  print(this_flip)
  if(this_flip == "Heads"){
    head_count <- head_count + 1
  }else{
    head_count <- 0
  }
  if(head_count == 3){
    three_heads <- TRUE
  }

}
```

Loading and Saving Files

Everything we have done so far has been run from the computer’s random access memory (RAM) which

is transient - as soon as the application is closed, or the PC is turned off it will vanish.

Most of the time we want to save and load files for use later on. Likewise, most of the data that we access will supplied either as a text file, or as some other file type (e.g. Nifti files for neuroimaging).

We're going to take a look at loading and saving text files.

Computer-Readable text files

You may have encountered spreadsheet formats such as `.xlsx` files before. These are typically much more complex then they need to be - containing information about text formatting, borders, pivot tables, etc. They are also highly variable and as a result can be hard for computers to read. We are going to be working a simpler format called delimited files which are just text files that represent rows and columns of a spreadsheet using newlines and a specified delimiter character.

To keep things simple, we're going to use commas as delimiters. The file extension for these is `.csv` (comma separated values). You can use other characters, such as semicolons or tabs, and in some cases this is preferable if your data is likely to contain commas.

In the following we're going to load the sleep data-set we looked at previously. We're then going to use the `write.csv()` function to save a copy of it to the hard drive. We're then going to remove the `sleep` dataframe using the `rm()` function, and then reload the file from our drive.

We don't currently know where our data will be saved. This location is known as the working directory, and we can get it using the `getwd()` function (short for get working directory).

Find out the destination below:

If this is not pointing at your OneDrive then it will probably be deleted when you log out of the university computer.

There are a couple of things we can do about this - the first is to use `setwd()` to set our working directory to a safer location.

The second, and more preferable, is to set up a project in RStudio.

Got to File -> New Project.

Now we can start saving things to our drive without worrying about them being deleted.

```
data(sleep)
head(sleep)

write.csv(sleep, file = 'sleep.csv', row.names = FALSE)

rm(sleep)

sleep <- read.csv(file= 'sleep.csv')
head(sleep)
```